

حلقه for در پایتون

حلقه for در پایتون برای تکرار کردن کاری در یک توالی («لیست» (List)، «تاپل» (Tuple)، «رشته» (String)) یا دیگر اشیای قابل تکرار، مورد استفاده قرار می‌گیرد. تکرار کردن کاری در یک توالی، «پیمایش» (Traversal) «نامیده می‌شود».

```
1 for val in sequence:
2     Body of for
```

مثالی از حلقه‌ها در پایتون

در قطعه کد زیر، مثالی از یک حلقه for در پایتون را مشاهده می‌کنید.

```
1 # Program to find the sum of all numbers stored in a List
2
3 # List of numbers
4 numbers = [6, 5, 3, 8, 4, 2, 5, 4, 11]
5
6 # variable to store the sum
7 sum = 0
8
9 # iterate over the list
10 for val in numbers:
11     sum = sum+val
12
13 # Output: The sum is 48
14 print("The sum is", sum)
```

خروجی حاصل از اجرای برنامه بالا، به صورت زیر خواهد بود.

```
The sum is 48
```

تابع range() در پایتون

می‌توان یک توالی از اعداد را با استفاده از تابع range() تولید کرد (range(10) ، اعداد از ۰ تا ۹ را تولید می‌کند (ده عدد)). همچنین، می‌توان سائز شروع، پایان و گام را به عنوان range(start,stop,step size) تعریف کرد.

```
1 # Output: range(0, 10)
2 print(range(10))
3
4 # Output: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
5 print(list(range(10)))
6
7 # Output: [2, 3, 4, 5, 6, 7]
8 print(list(range(2, 8)))
9
10 # Output: [2, 5, 8, 11, 14, 17]
11 print(list(range(2, 20, 3)))
```

می‌توان از تابع range() برای حلقه‌ها به منظور تکرار کردن یک توالی از اعداد استفاده کرد. این تابع را می‌توان با تابع len() برای تکرار کردن کاری در یک توالی با استفاده از اندیس‌دهی، ترکیب کرد. در ادامه، مثالی در همین رابطه ارائه شده است.

```
1 # Program to iterate through a list using indexing
2
3 genre = ['pop', 'rock', 'jazz']
4
5 # iterate over the list using index
6 for i in range(len(genre)):
7     print("I like", genre[i])
```

خروجی حاصل از اجرای برنامه بالا، به صورت زیر خواهد بود.

```
1 I like pop
2 I like rock
3 I like jazz
```

حلقه for else

یک حلقه for، می‌تواند یک بلوک else انتخابی نیز داشته باشد. بخش else، در صورتی اجرا می‌شود که عناصر توالی مورد استفاده از حلقه for به پایان برسند. از عبارت break می‌توان برای متوقف کردن حلقه for نیز استفاده کرد. در چنین شرایطی، بخش else نادیده انگاشته می‌شود. بنابراین، قسمت else حلقه for، در صورتی که هیچ خطایی وجود نداشته باشد اجرا می‌شود. در ادامه، مثالی برای حلقه for همراه با else ارائه شده است.

```
1 digits = [0, 1, 5]
2
3 for i in digits:
4     print(i)
5 else:
6     print("No items left.")
```

خروجی حاصل از اجرای برنامه بالا، به صورت زیر خواهد بود.

```
1 0
2 1
3 5
4 No items left.
```

در اینجا، حلقه for، عناصر لیست را تا هنگامی که حلقه متوقف شود، پرینت می‌کند. هنگامی که حلقه for متوقف شد، بلوک کد موجود در else اجرا و پرینت می‌شود.

```

letters = ['a', 'b', 'c', 'd']
print(letters)

upper_letters = []
for letter in letters:
    result = letter.upper()
    upper_letters.append(result)

print(upper_letters)

```

```

$python3 main.py
['a', 'b', 'c', 'd']
['A', 'B', 'C', 'D']

```

در قطعه کد زیر دقیقاً همان منطق را مشاهده می‌کنیم؛ به جز این که این کار تنها در یک خط با استفاده از خلاصه لیست مقدماتی پایتون صورت می‌گیرد:

```

letters = ['a', 'b', 'c', 'd']
rint(letters)

upper_letters = [x.upper() for x in letters]

print(upper_letters)

```

```

$python3 main.py
['a', 'b', 'c', 'd']
['A', 'B', 'C', 'D']

```

توضیح بیشتر با مثال

در مثال فوق لیستی به نام `Letters` ایجاد می‌شود. این لیست حروف کوچک «a»، «b»، «c» و «d» را در خود جای داده است. فرض کنید می‌خواهید همه این حروف را به حالت بزرگ تبدیل کنید. بدون وجود خلاصه لیست باید یک لیست جدید ایجاد کنید تا نتیجه را ذخیره نمایید، این لیست را `Upper_letters` می‌نامیم، سپس حلقه‌ای تعریف می‌کنیم تا همه عناصر موجود در آرایه `letters` را بررسی کرده و آن‌ها را به حروف بزرگ تبدیل کند و در لیست `result` ذخیره کند. البته این ذخیره‌سازی اختیاری است؛ اما بهتر است که بدین ترتیب عمل شود. سپس حروف بزرگ را به لیست جدید با استفاده از [متد `append` در پایتون](#) اضافه می‌کنیم. می‌بینید که کار زیادی برای انجام دادن وجود دارد.